Aprendizaje por refuerzo[1]

Gallego Nicolás, Juan Diego
jdiego.gallego@um.es

Salgueiro Santacreu, Rafel
rafel.salgueiro@um.es

Vera López, Óscar
oscar.veral@um.es

15 de marzo de 2026

[1]Máster en Inteligencia Artificial. Universidad de Murcia.

Resumen

En este trabajo se estudia el aprendizaje por refuerzo desde dos perspectivas complementarias.

En primer lugar, se aborda el problema del bandido de k-brazos, un escenario estático que permite analizar en profundidad el dilema exploración-explotación. Se comparan tres familias de algoritmos: ϵ-*greedy* (con y sin decaimiento), UCB y Softmax; sobre bandidos con distribuciones de Bernoulli, binomial y normal. Para la evaluación se utilizan las métricas de arrepentimiento acumulado y tasa de selección del brazo óptimo. Los resultados muestran que UCB destaca por su rapidez de convergencia cuando se dispone de información sobre la varianza del problema, mientras que ϵ-*greedy* con decaimiento ofrece un comportamiento más robusto y generalista.

En segundo lugar, se extiende el análisis a entornos con secuencialidad y estado, formalizados como Procesos de Decisión de Markov. Se evalúan métodos tabulares clásicos (Monte Carlo y diferencias temporales) en un entorno discreto de cuadrícula, y métodos de aproximación de función (SARSA semi-grediente y Deep Q-Learning) en el entorno continuo Flappy Bird. Se estudia el impacto de técnicas como el *replay buffer*, la red objetivo y el *reward shaping* sobre la estabilidad y la calidad del aprendizaje.

Índice general

1. Problema del Bandido de k-brazos **1**

1.1. Introducción . . . 1

1.2. Desarrollo . . . 1

1.3. Algoritmos . . . 2

1.3.1. Familia ϵ-*greedy* . . . 2

1.3.2. UCB . . . 3

1.3.3. Familia *Softmax* . . . 3

1.3.4. Comparativa entre familias . . . 3

1.4. Experimentos . . . 3

1.4.1. Bandido de Bernoulli . . . 3

1.4.2. Bandido Binomial . . . 4

1.4.3. Bandido normal . . . 5

1.5. Conclusiones . . . 6

2. Aprendizaje en Entornos Complejos **8**

2.1. Introducción . . . 8

2.2. Desarrollo . . . 8

2.2.1. Métodos tabulares . . . 8

2.2.2. Métodos continuos . . . 9

2.2.3. Entornos . . . 9

2.3. Algoritmos . . . 9

2.3.1. Monte Carlo . . . 9

2.3.2. SARSA . . . 10

2.3.3. Q-Learning . . . 10

2.3.4. SARSA Semi-Gradiente . . . 11

2.3.5. Deep Q-Learning (DQN) . . . 11

2.4. Métodos Tabulares . . . 12

2.4.1. Entorno Simple Grid 12
2.4.2. Configuración Experimental 12
2.4.3. Métricas de Desempeño y Resultados 12
2.4.4. Análisis 13
2.5. Métodos Aproximados 13
2.5.1. Entorno Flappy Bird 13
2.5.2. Configuración Experimental 13
2.5.3. Métricas de Desempeño y Resultados 14

Capítulo 1

Problema del Bandido de k-brazos

1.1. Introducción

En varios aspectos de la vida cotidiana un agente debe tomar decisiones repetidas bajo incertidumbre con el objetivo de maximizar su beneficio acumulado: selección de tratamientos médicos, personalización de contenidos en portales digitales... El **problema del bandido multibrazo** (o de k-brazos) es un escenario clásico para la aplicación de aprendizaje por refuerzo [1] que captura y modela matemáticamente este tipo de problema. El paradigma planteado es un agente que debe seleccionar secuencialmente acciones (brazos) de extracción para maximizar las recompensas acumuladas a lo largo del tiempo. Se asume que las diferentes recompensas siguen distribuciones de probabilidad ocultas y que el resultado de cada acción es independiente de las anteriores.

El agente puede almacenar en cada instante diferentes estadísticos que guíen sus futuras elecciones. Según su método de toma de decisiones, distinguimos dos comportamientos:

- **Exploración**: el agente selecciona un brazo (no necesariamente el óptimo) para aumentar su información sobre el mismo y aprovechar el nuevo dato para reforzar sus estadísticos. Un comportamiento puramente exploratorio obtiene una recompensa media igual a la media de las recompensas individuales de los brazos.
- **Explotación**: el agente selecciona el brazo que considera que ofrece mejores recompensas. Esta selección se hace en general según la media. Un comportamiento puramente explotador obtendrá una recompensa media igual a la media de uno de los brazos. Dependiendo de las elecciones iniciales, es posible que este brazo no sea el óptimo.

El **Dilema Exploración-Explotación** es el núcleo central del problema y todos los algoritmos que se plantean en este paradigma tratan de forma diferente de llegar a un equilibrio que maximice las recompensas en un horizonte temporal indeterminado.

En este capítulo comparamos tres familias de algoritmos (***ϵ-greedy***, **UCB** y ***Softmax***) sobre tres bandidos con distribuciones de probabilidad diferentes (**Bernoulli**, **binomial** y **normal**), con el objetivo de determinar los mejores agentes para cada problema o comprobar si existe algún método claramente superior a los demás en todos los escenarios.

El capítulo se organiza presentando primero la definición formal del problema y las métricas empleadas, seguido de los pseudocódigos de cada algoritmo, los resultados experimentales para cada tipo de bandido, y finalmente las conclusiones del estudio.

Todo el código desarrollado en este apartado se puede encontrar en el repositorio de GitHub https://github.com/rafelsalgueiro/GallegoSalgueiroVera. El esqueleto del mismo ha sido proporcionado por el profesor de la asignatura. Los experimentos han sido desarrollados en los *notebooks* que se encuentran dentro de la carpeta 'k_brazos'.

1.2. Desarrollo

El problema se enmarca como un proceso de toma de decisiones secuencial donde un agente interactúa con un entorno compuesto por múltiples opciones independientes. Sea t un instante de tiempo dado y $\mathcal{A} = \{1, ..., K\}$ el conjunto de las posibles acciones. Definimos:

- Acción: $A_t \in \mathcal{A}$.
- Recompensa: $R_t \in \mathbb{R}$.

Cada acción a tiene asociada una recompensa esperada que llamaremos μ_a:

$$\mu_a = \mathbb{E}[R_t \mid A_t = a]$$

El objetivo principal del agente es maximizar la suma total de las recompensas obtenidas a lo largo de los T pasos de tiempo. Para formular nuestras métricas es útil definir de antemano el brazo óptimo y su valor:

- Brazo óptimo: $a^* = argmax_a\{\mu_a\}$
- Recompensa esperada óptima: $q^* = max_a\{\mu_a\}$

Para evaluar qué tan bien funciona una estrategia, utilizamos las siguientes métricas [3]:

- **Recompensa media acumulada:** el valor esperado de la suma de recompensas hasta el paso T.

$$\mathbb{E}\left[\sum_{t=1}^{T} R_t\right]$$

- **Tasa de elección del brazo óptimo:** la probabilidad de que el algoritmo haya aprendido a seleccionar la mejor acción posible en el instante T.

$$\mathbb{P}\left[A_T = a^*\right]$$

- **Arrepentimiento acumulado:** diferencia entre la recompensa total que el agente habría obtenido si hubiera

conocido y elegido siempre el brazo óptimo y la recompensa que obtuvo.

$$L_T = T \times q^* - \sum_{t=1}^{T} R_t$$

Las tres métricas son complementarias, pero el arrepentimiento acumulado es la más informativa: equivale a medir la distancia entre el comportamiento del agente y el de un agente con conocimiento perfecto que explota siempre el brazo óptimo. Un arrepentimiento que crece sublinealmente con T indica que el agente está aprendiendo de forma efectiva. La **cota de Lai-Robbins** [2] establece un límite inferior teórico sobre el arrepentimiento acumulado esperado. Formalmente, para un horizonte temporal T suficientemente grande, ningún algoritmo puede hacer mejor que

$$\mathbb{E}[L_T] \geq \sum_{a:\mu_a<\mu^*} \frac{\mu^* - \mu_a}{KL(\mu_a||\mu^*)} \cdot \ln T,$$

donde $KL(\mu_a||\mu^*)$ es la divergencia de Kullback-Leibler entre la distribución del brazo a y la del brazo óptimo, que mide cuán difíciles son de distinguir estadísticamente. Esta expresión implica que el arrepentimiento de cualquier algoritmo razonable crece al menos de forma logarítmica con el número de pasos. En la práctica, utilizamos esta cota como referencia visual en las gráficas de arrepentimiento acumulado para valorar cuán cerca están los algoritmos evaluados del comportamiento asintóticamente óptimo.

Por último, los tres tipos de bandidos estudiados se diferencian en la distribución de probabilidad que rige las recompensas de cada brazo:

- **Bandido de Bernoulli:** es el más sencillo. Las recompensas son binarias (0/1) y la recompensa del brazo a sigue una distribución de Bernoulli con un parámetro $p_a \in [0, 1]$, que representa la probabilidad de éxito.

$$R_t \sim Be(p_a)$$

- **Bandido binomial:** las recompensas son discretas ($\{0..n\}$) y la recompensa del brazo a sigue una distribución binomial de n ensayos y probabilidad p_a.

$$R_t \sim Binom(n, p_a)$$

- **Bandido normal:** las recompensas son continuas y la recompensa del brazo a sigue una distribución normal con media μ_a y desviación típica σ_a.

$$R_t \sim N(\mu_a, \sigma_a)$$

Los algoritmos que empleamos están descritos en la siguiente sección [3]. Al tratarse de una práctica de introducción a un problema clásico, no pretendemos hacer ninguna innovación técnica sino un trabajo exploratorio para comprender mejor estos algoritmos.

1.3. Algoritmos

1.3.1. Familia *ϵ-greedy*

La primera familia que probamos es la llamada *ϵ-greedy*. En su forma clásica viene controlada por un parámetro $\epsilon \in [0, 1]$ que determina la probabilidad de exploración. En el algoritmo 1, en cada elección se decide con una probabilidad $p = \epsilon$ escoger un brazo de forma uniformemente aleatoria (exploración) y con una probabilidad $p = 1 - \epsilon$ el brazo de mayor valor medio (explotación).

Algorithm 1 *ϵ-greedy*

Require: $\epsilon \in (0, 1)$, K, T
Inicialización:
for $a = 1 \dots K$ **do**
  $Q(a) \leftarrow 0$ ▷ Estimación de la recompensa
  $N(a) \leftarrow 0$ ▷ Contador de elecciones del brazo a
end for
Interacción con el entorno:
for $t = 1 \dots T$ **do**
  $p \leftarrow$ Muestreo $Unif((0, 1))$
  if $p < \epsilon$ **then**
    $A_t \leftarrow$ Muestreo $Unif(\{1, \dots, K\})$ ▷ Exploración
  else
    $A_t \leftarrow \arg\max_a Q(a)$ ▷ Explotación
  end if
  $R_t \leftarrow$ Ejecutar A_t
  Actualización de estimaciones:
  $N(A_t) \leftarrow N(A_t) + 1$
  $Q(A_t) \leftarrow Q(A_t) + \frac{1}{N(A_t)} (R_t - Q(A_t))$
end for

También empleamos una variante con decaimiento del parámetro ϵ. La motivación es dedicar las iteraciones iniciales a una exploración exhaustiva y reducir progresivamente la aleatoriedad conforme el agente acumula conocimiento, favoreciendo la explotación en fases avanzadas. La actualización de ϵ sigue la regla $\epsilon_t = \frac{\epsilon_0}{1+\lambda t}$, donde $\lambda > 0$ controla la velocidad de decaimiento.

Algorithm 2 *ϵ-greedy* con decaimiento proporcional

Require: $\epsilon_0 \in (0, 1)$, $\lambda > 0$ K, T
Inicialización:
for $a = 1 \dots K$ **do**
  $Q(a) \leftarrow 0$ ▷ Estimación de la recompensa
  $N(a) \leftarrow 0$ ▷ Contador de elecciones del brazo a
  $\epsilon \leftarrow \epsilon_0$
end for
Interacción con el entorno:
for $t = 1 \dots T$ **do**
  $p \leftarrow$ Muestreo $Unif((0, 1))$
  if $p < \epsilon$ **then**
    $A_t \leftarrow$ Muestreo $Unif(\{1, \dots, K\})$ ▷ Exploración
  else
    $A_t \leftarrow \arg\max_a Q(a)$ ▷ Explotación
  end if
  $R_t \leftarrow$ Ejecutar A_t
  Actualización de estimaciones:
  $N(A_t) \leftarrow N(A_t) + 1$
  $Q(A_t) \leftarrow Q(A_t) + \frac{1}{N(A_t)} (R_t - Q(A_t))$
  $\epsilon \leftarrow \frac{\epsilon_0}{1+\lambda t}$
end for

1.3.2. UCB

Esta segunda familia (*Upper Confidence Bound*) introduce el concepto de exploración dirigida basada en el principio de 'optimismo frente a la incertidumbre'. En lugar de separar rígidamente las dos fases, UCB1 (algoritmo 3) evalúa el potencial de cada brazo como una ponderación del valor estimado y un margen de confianza que representa la incertidumbre acumulada sobre ese brazo. La fórmula de selección es:

$$UCB1(a) = Q(a) + c\sqrt{\frac{\ln t}{N(a)}}$$

El parámetro $c > 0$ regula el peso otorgado a la incertidumbre. Un valor alto de c favorece la exploración: brazos poco visitados recibirán puntuaciones UCB elevadas independientemente de su rendimiento observado. Un valor bajo de c concentra la selección en los brazos con mayor valor estimado, priorizando la explotación. A diferencia de ϵ-*greedy*, UCB1 no explora de forma aleatoria sino dirigida, favoreciendo sistemáticamente los brazos sobre los que existe mayor incertidumbre.

Algorithm 3 Algoritmo UCB1 (Upper Confidence Bound)

Require: $c > 0$ (típicamente $c = \sqrt{2}$), K, T
Inicialización:
for $a = 1 \dots K$ **do**
  $R_a \leftarrow$ Ejecutar a
  $Q(a) \leftarrow R_a$ ▷ Jugar cada brazo una vez
  $N(a) \leftarrow 1$ ▷ Contador de selecciones del brazo
end for
Interacción con el entorno:
for $t = K+1 \dots T$ **do**
  for $a = 1 \dots K$ **do**
    $UCB(a) \leftarrow Q(a) + c\sqrt{\frac{\ln t}{N(a)}}$
  end for
  $A_t \leftarrow \arg\max_a UCB(a)$ ▷ (romper empates al azar)
  $R_t \leftarrow$ Ejecutar A_t
  Actualización de estimaciones:
  $N(A_t) \leftarrow N(A_t) + 1$
  $Q(A_t) \leftarrow Q(A_t) + \frac{1}{N(A_t)}(R_t - Q(A_t))$
end for

1.3.3. Familia *Softmax*

La estrategia *Softmax* (algoritmo 4) pondera las probabilidades de exploración según las estimaciones de recompensa actuales [6]. En lugar de elegir siempre la mejor acción, su elección se basa en una distribución de probabilidad dada por la función *Softmax*. Esta probabilidad viene controlada por un parámetro al que se le conoce comúnmente por 'temperatura' (τ). Así, la probabilidad de escoger el brazo a en el instante t se define como:

$$\mathbb{P}(A_t = a) = \frac{e^{q_a/\tau}}{\sum_{b=1}^{K} e^{q_b}/\tau}$$

- Si $\tau \to \infty$: las exponenciales tienden a 1, por lo que las acciones serán equiprobables (comportamiento totalmente exploratorio).
- Si $\tau \to 0$: la diferencia entre los valores se magnifica en extremo, por lo que el brazo con mayor q_a se lleva el 100 % de la probabilidad (explotación pura).

Algorithm 4 Algoritmo Softmax (Exploración Boltzmann)

Require: $\tau > 0$, K, T
Inicialización:
for $a = 1 \dots K$ **do**
  $Q(a) \leftarrow 0$ ▷ Estimación del valor esperado
  $N(a) \leftarrow 0$ ▷ Contador de elecciones del brazo
end for
Interacción con el entorno:
for $t = 1 \dots T$ **do**
  for $a = 1 \dots K$ **do**
    $\pi(a) \leftarrow \frac{e^{Q(a)/\tau}}{\sum_{b=1}^{K} e^{Q(b)/\tau}}$ ▷ Softmax
  end for
  $A_t \leftarrow$ Muestrear de $\pi(\{1 \dots K\})$
  $R_t \leftarrow$ Ejecutar A_t
  Actualización de estimaciones:
  $N(A_t) \leftarrow N(A_t) + 1$
  $Q(A_t) \leftarrow Q(A_t) + \frac{1}{N(A_t)}(R_t - Q(A_t))$
end for

1.3.4. Comparativa entre familias

ϵ-*greedy* es el enfoque más sencillo, pues separa ambas fases de forma rígida. UCB refina esta idea explorando de forma informada, priorizando los brazos de mayor incertidumbre y garantizando propiedades teóricas de arrepentimiento sublineal [5]. *Softmax*, por su parte, integra el conocimiento acumulado en una distribución de probabilidad, ofreciendo una transición más suave entre exploración y explotación, pero a costa de una mayor sensibilidad a la escala de las recompensas y al ajuste del parámetro τ. En la siguiente sección evaluamos empíricamente estas diferencias sobre los tres tipos de bandidos.

1.4. Experimentos

1.4.1. Bandido de Bernoulli

Configuración experimental

Se modela un bandido de cinco brazos con recompensas de distribución Bernoulli y probabilidades de éxito $p \in \{0{,}1, 0{,}3, 0{,}5, 0{,}7, 0{,}9\}$. Las recompensas son binarias, lo que las convierte en la señal de menor varianza de los tres bandidos estudiados. Se simulan 1000 iteraciones promediadas sobre 500 ejecuciones independientes. Se prueban variantes de las tres familias con distintos niveles de exploración, y se añade un experimento adicional con brazos de probabilidades muy similares ($p \in \{0{,}4, 0{,}42, 0{,}45, 0{,}48, 0{,}5\}$) para estudiar el comportamiento de los algoritmos en condiciones más complejas.

Resultados

Algoritmo	*Regret* Final	% Sel. Óptima
ϵ-*greedy* ($\epsilon = 0{,}1$)	56.22	90.20 %
ϵ-*decay* ($\epsilon = 0{,}1$)	-	≈99 %
UCB1 ($c = 0{,}5$)	13.23	98.60 %
Softmax ($\tau = 0{,}1$)	74.40	76.80 %

Tabla 1.1: Desempeño final promedio en el bandido Bernoulli.

Análisis

- **Familia ϵ-*greedy***. El agente puramente explotador ($\epsilon = 0$) se comporta como cabría esperar. Obtiene una tasa de acierto constante igual a $1/n$. Los valores bajos de épsilon $\epsilon = 0{,}01$ muestran curvas de descubrimiento del óptimo lentas. El valor $\epsilon = 0{,}1$ ofrece el mejor equilibrio entre exploración y explotación, convergiendo de forma rápida y alcanzando un 90,20 % de probabilidad de selección del brazo óptimo. No obstante, su *regret* sigue creciendo linealmente al dedicar siempre el 10 % de las iteraciones a la exploración aleatoria. $\epsilon = 0{,}3$ aprende rápidamente al inicio, pero su tasa de selección del brazo óptimo se estanca igual de rápido alrededor del 76 %. La variante con decaimiento alcanza un 99 % de selección óptima al combinar una exploración intensiva inicial con una explotación creciente.
- **Familia UCB**. Con $c = 0{,}5$, UCB1 alcanza un 98.6 % de tasa de selección del brazo óptimo y un *regret* final de tan solo 13.24, siguiendo de cerca la cota teórica de Lai-Robbins. Este escenario de poca varianza es especialmente favorable para UCB: con pocas muestras por brazo, el intervalo de confianza se estrecha rápidamente.
- **Familia *Softmax***. Con $\tau = 0{,}1$ *Softmax* obtiene un 76,80 % de selección óptima y el mayor *regret* de la comparativa (74,40). El valor $\tau = 0{,}01$ colapsa de forma casi al igual que $\epsilon = 0$, mientras que $\tau = 0{,}5$ y $\tau = 1{,}0$ acumulan arrepentimiento creciente por exceso de exploración.

Figura 1.1: Selección óptima para brazos de Bernouilli.

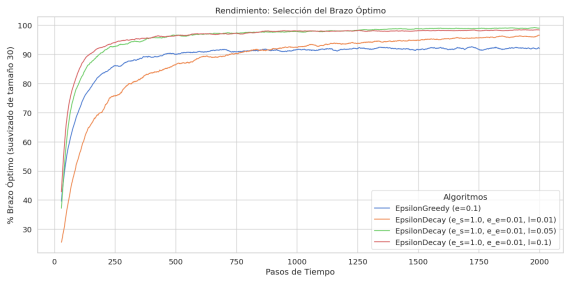

Figura 1.2: Selección óptima para brazos de Bernouilli con Epsilon-Decay.

En el experimento con brazos de probabilidades similares ($p \in [0{,}4, 0{,}42, 0{,}45, 0{,}48, 0{,}5]$), la dificultad estadística del problema aumenta: la constante de Lai-Robbins se dispara de 3,12 a 46,17. En este contexto, UCB1 mantiene una leve ventaja sobre ϵ-greedy en la tasa de selección óptima, mientras que *Softmax* acumula el mayor arrepentimiento.

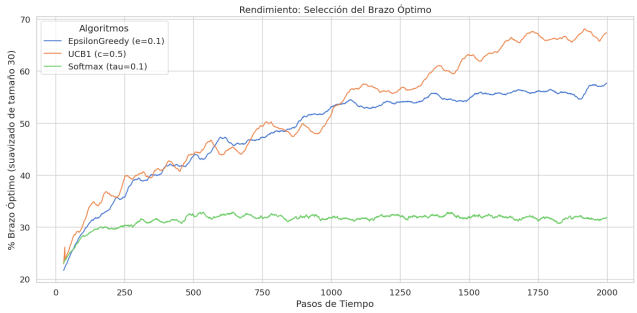

Figura 1.3: Selección óptima en brazos de Bernouilli similares.

1.4.2. Bandido Binomial

Configuración experimental

Se modela un bandido de 10 brazos con recompensas de distribución Binomial de $n = 10$ ensayos y probabilidades de éxito equiespaciadas en el intervalo $[0{,}1, 0{,}9]$, de modo que el brazo óptimo tiene $p = 0{,}9$. Dado que las recompensas se normalizan dividiendo por n, el valor esperado de cada brazo coincide directamente con su probabilidad de éxito. Las recompensas son discretas (enteros en $\{0, \ldots, n\}$ normalizados a $[0, 1]$). Para evaluar los algoritmos (ϵ-*greedy*, UCB1 y *Softmax*), se simulan 1000 iteraciones promediadas sobre 500 ejecuciones independientes. Se utiliza una semilla fija para asegurar la reproducibilidad del experimento.

Resultados

Algoritmo	*Regret* Final	% Sel. Óptima
ϵ-*greedy* (Decay)	25.50	95.80 %
UCB1 ($c = 0{,}5$)	37.86	94.80 %
Softmax ($\tau = 0{,}1$)	100.18	48.60 %

Tabla 1.2: Desempeño promedio en el bandido Binomial.

Análisis

- **Familia ϵ-*greedy***. Los agentes con ϵ pequeño (0,01) mejoran progresivamente pero de forma lenta, mientras que $\epsilon = 0{,}1$ ofrece el mejor equilibrio entre exploración y explotación dentro de los métodos estáticos. Sin embargo, incluso en este caso el *regret* sigue creciendo linealmente, penalizado por la exploración permanentemente. La variante con decaimiento resuelve esta limitación logrando el menor arrepentimiento acumulado ($\approx 25{,}50$) y la mayor tasa de selección óptima (95,80 %).

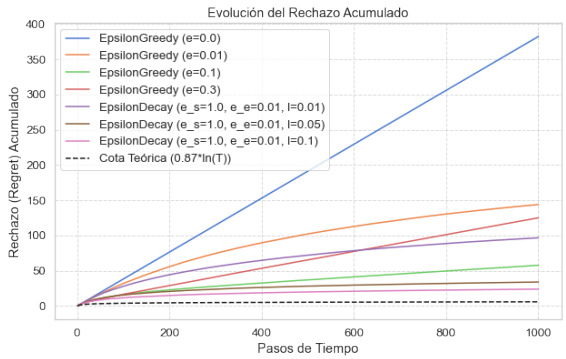

Figura 1.4: Rechazo acumulado en bandido binomial con ϵ-*greedy*.

- **Familia UCB**. Los agentes UCB demuestran una solidez notable y una menor sensibilidad a la calibración del parámetro c que ϵ-*greedy* con respecto a ϵ. En contraste con los métodos ϵ-*greedy*, UCB presenta una convergencia con crecimiento logarítmico del arrepentimiento. El valor $c = 0{,}5$ resulta ser el más eficiente en este entorno, alcanzando valores cercanos al óptimo rápidamente y superando el 90 % de selección óptima al final del horizonte. Valores más altos de c introducen un exceso de exploración que retrasa la convergencia sin beneficio apreciable.

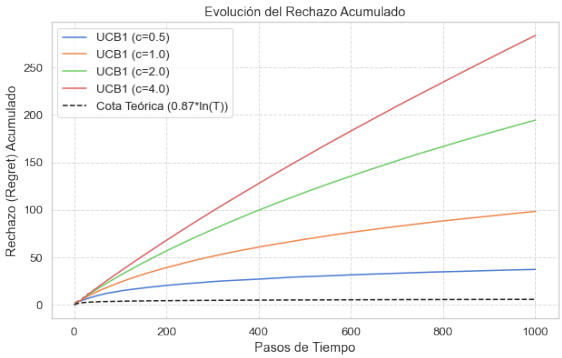

Figura 1.5: Rechazo acumulado en bandido binomial con UCB.

- **Familia *Softmax***. Con $\tau = 0{,}1$, *Softmax* logra la mayor velocidad de subida inicial entre todos los algoritmos, pero se estanca entorno al 48,60 % de selección óptima. Los valores $\tau = 0{,}5$ y $\tau = 1{,}0$ presentan incluso peores resultados por exceso de exploración, mientras que $\tau = 0{,}01$ colapsa de forma casi determinista desde el inicio al igual que ϵ-*greedy* con $\epsilon = 0$.

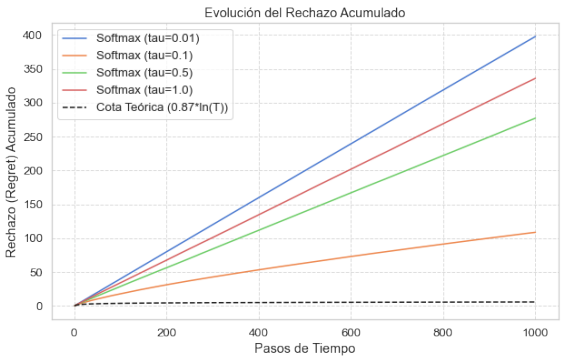

Figura 1.6: Rechazo acumulado en bandido binomial con *Softmax*.

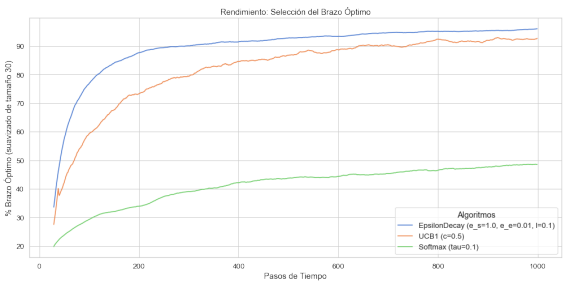

Figura 1.7: Tasa de selección del brazo óptimo en los mejores agentes de cada familia para el bandido binomial.

1.4.3. Bandido normal

Configuración experimental

Se utilizan dos bandidos con distribuciones normales con medias $\mu_a \in (0, 10)$ y que difieren únicamente en su desviación típica: el **bandido sencillo** con $\sigma = 1$ (un 10 % del rango de medias) y el **bandido complejo** con $\sigma = 5$ (un 50 % del rango de medias). Nuestra hipótesis es que una mayor varianza ralentizará el aprendizaje y confundirá a los agentes, dado que las distribuciones de recompensa de los brazos se solaparán en mayor medida. Para garantizar la solidez estocástica, el desempeño de los algoritmos se evalúa a lo largo de un horizonte de 1000 iteraciones, promediado sobre 500 ejecuciones independientes. Se prueban variantes de las tres familias de algoritmos con parámetros que favorecen en mayor y menor medida la exploración.

Cabe destacar que para la familia *Softmax* se han utilizado temperaturas en el rango $\tau \in [0{,}5, 5]$ en lugar del valor $\tau = 0{,}1$ usado en bandidos anteriores. Esto se debe a que, con recompensas en el rango $(0, 10)$, valores de τ pequeños provocan desbordamientos numéricos en el cálculo de las exponenciales ($e^{9/0{,}1} \gg 2^{63}$).

Resultados

Algoritmo	***Regret*** **(sencillo)**	**% Óptima (sencillo)**	***Regret*** **(complejo)**	**% Óptima (complejo)**
ϵ-Greedy ($\epsilon = 0{,}1$)	$\sim$180	$\sim$75 %	$\sim$500	$\sim$55 %
ϵ-Decay ($\lambda = 0{,}01$)	$\sim$260	$\sim$89 %	$\sim$480	$\sim$70 %
ϵ-Decay ($\lambda = 0{,}1$)	$\sim$110	$\sim$72 %	$\sim$490	$\sim$65 %
UCB1 ($c = 1$)	$\sim$60	> 90 %	$\sim$420	< 60 %
UCB1 ($c = 5$)	$\sim$200	$\sim$65 %	$\sim$350	$\sim$85 %
Softmax ($\tau = 1$)	> 400	$\sim$46 %	> 500	$\sim$43 %

Tabla 1.3: Desempeño final promedio de los algoritmos en el bandido Normal (valores aproximados en la iteración 1000).

Análisis

- **Familia ϵ-*greedy*** Los algoritmos con exploración baja ($\epsilon = 0{,}005, 0{,}01$) y decaimiento lento ($\lambda = 0{,}01$) son los que mayor arrepentimiento acumulan en las primeras iteraciones, sin identificar el brazo óptimo de forma consistente, llegando a tasas inferiores al 60 % en el bandido sencillo y al 50 % en el complejo. El mecanismo de decaimiento mejora notablemente los resultados: un decaimiento rápido ($\lambda = 0{,}1$) minimiza el arrepentimiento acumulado en el bandido sencillo ($\approx 110 puntos$), mientras que un decaimiento lento ($\lambda = 0{,}01$) alcanza la mayor tasa de selección óptima a largo plazo ($\approx$ 89 %), a costa de acumular más arrepentimiento en las fases iniciales.
- **Familia UCB**. Los agentes UCB obtienen en general los mejores resultados, pero demuestran una notable sensibilidad a la varianza del problema. En el bandido sencillo, UCB con $c = 1$ se consolida como el ganador indiscutible en todas las métricas: supera el 90 % de selección óptima y acumula el menor arrepentimiento, siguiendo de cerca la cota teórica de Lai-Robbins. Sin embargo, en el bandido complejo este mismo agente se estanca entorno a la iteración 200 y no supera el 60 % de selección ópti-

ma, confundiendo el brazo óptimo con el segundo mejor (cuyas medias difieren en solo 0,60 puntos). En este escenario, UCB con $c = 5$ se impone claramente: es el único agente que alcanza el 85 % de selección óptima y cuya curva de arrepentimiento presenta un punto de inflexión visible alrededor de la iteración 200, siendo el único que no se comporta como una función lineal al final del horizonte.

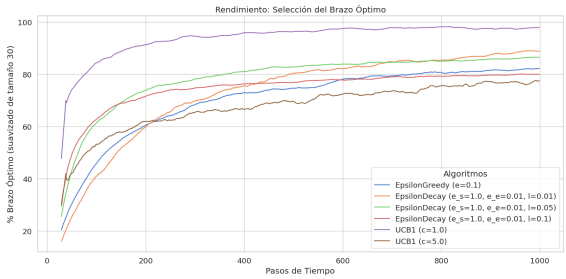

Figura 1.8: Selección del brazo óptimo en el bandido sencillo.

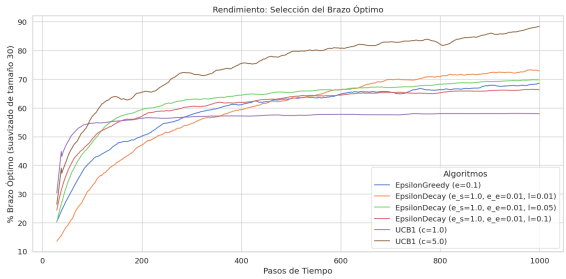

Figura 1.9: Selección del brazo óptimo en el bandido complejo.

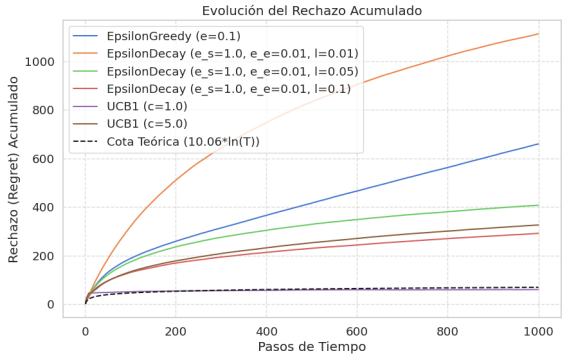

Figura 1.10: *Regret* acumulado en el bandido sencillo.

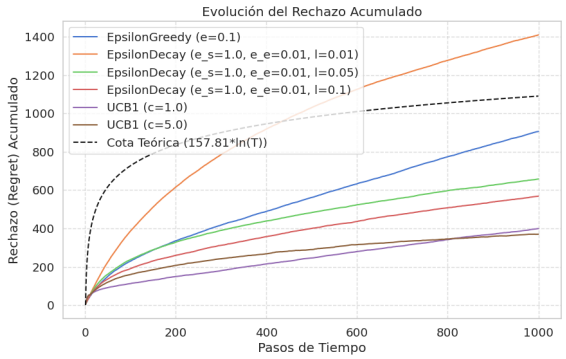

Figura 1.11: *Regret* acumulado en el bandido complejo.

- **Familia *Softmax***. Es la perdedora con amplia diferencia. El mejor agente ($\tau = 1$) es el único que no padece estancamiento temprano y mantiene una tendencia creciente en la iteración 1000, pero su tasa de selección óptima no supera el 46 % en el bandido sencillo ni el 43 % en el complejo. Los agentes con temperatura alta ($\tau \geq 2$) adoptan un comportamiento plenamente explotador con tasa de acierto por debajo del 40 %, mientras que el más explorador ($\tau = 0{,}5$) acumula un arrepentimiento que sigue una tendencia lineal sin signo de convergencia.

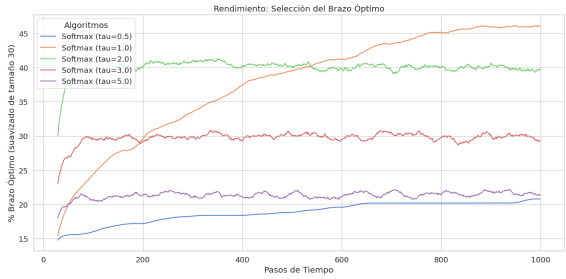

Figura 1.12: Tasa de selección del brazo óptimo en el bandido sencillo con estrategias *Softmax*.

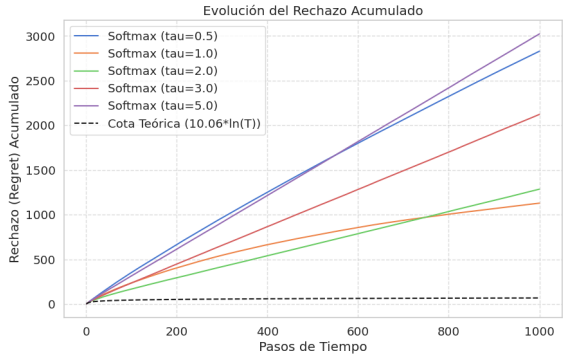

Figura 1.13: Arrepentimiento acumulado en el bandido sencillo con estrategias *Softmax*.

1.5. Conclusiones

A lo largo de este capítulo hemos evaluado tres familias de algoritmos sobre tres tipos de bandidos con distribuciones subyacentes distintas: Bernoulli, binomial y normal. Los resultados permiten extraer conclusiones tanto sobre el comportamiento individual de cada familia como sobre la influencia de la distribución de recompensas en la elección del algoritmo óptimo.

En primer lugar, concluimos que el tipo de distribución tiene un impacto directo y significativo en el rendimiento de los algoritmos. En el bandido de Bernoulli, las recompensas binarias ofrecen una señal clara y de baja varianza, lo que facilita la identificación del brazo óptimo: UCB1 con $c = 0{,}5$ alcanza un 98 % de selección óptima y un *regret* de solo 13,23. En el bandido binomial, la naturaleza discreta de las recompensas introduce mayor ruido, y Softmax colapsa completamente. En el bandido normal, la varianza es el factor determinante: con $\sigma = 1$, UCB1 con $c = 1$ domina con un *regret* de ≈ 60 y más del 90 % de selección óptima; con $\sigma = 5$, ese mismo agente se

Bandido	**Ganador (*regret*)**	**Ganador (% óptima)**
Bernoulli	UCB1 ($c = 0{,}5$)	UCB1 ($c = 0{,}5$)
Binomial	ϵ-Decay ($\lambda = 0{,}1$)	ϵ-Decay ($\lambda = 0{,}1$)
Normal ($\sigma = 1$)	UCB1 ($c = 1$)	UCB1 ($c = 1$)
Normal ($\sigma = 5$)	UCB1 ($c = 5$)	UCB1 ($c = 5$)

Tabla 1.4: Resultados según el tipo de bandido.

estanca por debajo del 60 % y es UCB con $c = 5$ quien toma el relevo con un 85 % de selección óptima y menor arrepentimiento acumulado (≈ 350). En resumen, a mayor varianza o complejidad en la distribución, más costoso es el proceso de aprendizaje y más necesaria es una exploración elevada. La tabla 1.4 siguiente sintetiza los ganadores por escenario.

La elección del algoritmo debe también considerar el horizonte de tiempo disponible. En horizontes cortos, UCB es la opción más clara: su convergencia es rápida y sus exploraciones son mucho más eficientes que las de ϵ-*greedy*. En horizontes largos, la ventaja de UCB se diluye y ϵ-*greedy* con decaimiento se convierte en una alternativa competitiva, especialmente en problemas de alta varianza donde el parámetro c de UCB es difícil de calibrar. *Softmax*, según nuestro estudio, no es recomendable en ningún horizonte temporal sin un proceso de ajuste previo exhaustivo.

Este estudio presenta varias limitaciones que abren líneas de trabajo futuras de interés. En primer lugar, todos los experimentos se han realizado con un número fijo de brazos. Variar K permitiría estudiar cómo escala el coste de exploración con la dimensionalidad del problema. En segundo lugar, hemos evaluado cada familia sobre un único tipo de bandido a la vez. Un análisis con distribuciones mixtas acercaría al experimento a escenarios reales. En tercer lugar, quedan fuera del estudio técnicas de mayor sofisticación teórica como el muestreo de Thompson [8], que ha demostrado resultados competitivos con UCB en la literatura [9], o métodos bayesianos que actualicen distribuciones de probabilidad completas sobre los brazos en lugar de simples estimaciones de media. Finalmente, sería interesante estudiar la robustez de los algoritmos ante bandidos no estacionarios, donde las distribuciones de recompensa cambian paulatinamente a lo largo del tiempo, escenario en el que ϵ-*greedy* con decaimiento podría verse especialmente perjudicado.

Capítulo 2

Aprendizaje en Entornos Complejos

2.1. Introducción

El problema del bandido de k-brazos trata principalmente sobre el dilema de explotación-exploración en un contexto estático y estacionario: el agente selecciona acciones cuyas recompensas dependen únicamente de la acción elegida, sin que el entorno cambie como consecuencia de ella. No obstante, la toma de decisiones real rara vez carece de contexto o consecuencias. En entornos complejos, las acciones de un agente no solo determinan la recompensa inmediata, sino que también provocan una transición hacia un nuevo estado del entorno. La recompensa ya no depende solo de la acción, sino del par estado-acción, y las decisiones presentes condicionan las oportunidades futuras. Para modelar esta secuencialidad es necesario formular el problema como un **Proceso de Decisión de Markov** (MDP).

Bajo este marco, el agente debe aprender mediante la interacción y la experiencia directa, actualizando su política para maximizar la recompensa acumulada futura. A diferencia de los métodos de planificación clásica que requieren conocer explícitamente la dinámica del entorno, todos los algoritmos estudiados en este capítulo son **libres de modelo** (*model-free*): el agente no tiene acceso previo a ese conocimiento y debe inferir su política únicamente a partir de las transiciones que observa al interactuar con el entorno.

Nuestro objetivo con este capítulo es evaluar el rendimiento, la estabilidad y la capacidad de convergencia de diversas técnicas de aprendizaje por refuerzo *model-free* en dos entornos diferenciados por la dimensión de sus estados. En primer lugar, analizaremos empíricamente **métodos tabulares** clásicos (Monte Carlo y Diferencias Temporales) aplicados a un entorno de control discreto (SimpleGrid), donde el espacio de estados es finito y manejable. Posteriormente, dado que en problemas continuos la cantidad de estados hace inviable la representación tabular, estudiaremos la integración de **aproximadores de funciones** (SARSA semi-gradiente y Deep Q-Learning) aplicados a un entorno complejo y continuo como es Flappy Bird.

El resto del capítulo se organiza de la siguiente manera: la sección 2 formaliza el contexto teórico y los enfoques utilizados; la sección 3 detalla los algoritmos implementados; la sección 4 presenta el diseño experimental, los resultados empíricos y el análisis crítico de los métodos tabulares; y la sección 5 hace lo propio con los métodos aproximados.

2.2. Desarrollo

El aprendizaje por refuerzo en entornos complejos se formaliza habitualmente mediante un **Proceso de Decisión de Markov** (MDP). Un MDP se define por la tupla $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$, donde $\mathcal{S}$ es el conjunto de estados posibles, $\mathcal{A}$ es el conjunto de acciones, $\mathcal{P}(s'|s,a)$ representa la probabilidad de transición al estado s' tras ejecutar la acción a en el estado s, $\mathcal{R}(s,a)$ es la función de recompensa esperada y $\gamma \in [0,1]$ es el factor de descuento. En cada instante de tiempo t, el agente observa un estado $S_t \in \mathcal{S}$, selecciona una acción $A_t \in \mathcal{A}$ siguiendo una política $\pi(a|s)$, y el entorno responde con un nuevo estado S_{t+1} y una recompensa R_{t+1}. El objetivo es encontrar una política óptima π_* que maximice el retorno esperado

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

El factor de descuento γ controla el horizonte temporal efectivo del agente: valores próximos a cero producen agentes cortoplacistas que priorizan la recompensa inmediata, mientras que valores próximos a uno hacen que el agente valore en mayor medida las consecuencias futuras.

2.2.1. Métodos tabulares

Para resolver el MDP cuando el espacio de estados y acciones es discreto y abarcable, se emplean métodos tabulares que estiman el valor exacto de la función de acción-valor $Q(s,a)$. En la literatura destacan dos enfoques principales:

- **Montecarlo (MC):** Aprende a partir de la experiencia generada en episodios completos. Actualiza las estimaciones de valor promediando los retornos empíricos G_t obtenidos al llegar a un estado terminal.
- **Diferencias Temporales (TD):** Aprende paso a paso sin esperar al final del episodio, basando su actualización en las estimaciones actuales de los estados siguientes (*bootstrapping*). Algoritmos como SARSA (*on-policy*) y Q-Learning (*off-policy*) pertenecen a esta familia.

Bajo condiciones estándar (visita repetida de todos los pares estado-acción y tasa de aprendizaje decreciente apropiada) ambos enfoques garantizan la convergencia a los valores exactos de Q^*, lo que los convierte en métodos teóricamente sólidos para espacios de estados finitos.

2.2.2. Métodos continuos

Cuando nos enfrentamos a problemas con espacios de estados continuos o de alta dimensionalidad, la representación tabular se vuelve inviable debido a limitaciones drásticas de memoria y a la incapacidad de generalizar la experiencia a estados no visitados de forma eficiente. Para superar esto se recurre a la **aproximación de funciones**. En lugar de una tabla, se utiliza una función parametrizada con pesos θ, de modo que $Q(s, a; \theta) \approx q_*(s, a)$. La optimización de estos pesos se realiza típicamente mediante técnicas de descenso del gradiente.

Sin embargo, esta extensión introduce un reto fundamental: las garantías de convergencia no se cumplen. Cuando la función aproximadora no es lineal (como una red neuronal profunda), la combinación del *bootstrapping*, la correlación temporal entre muestras consecutivas y la inestabilidad del objetivo puede producir divergencia. Esta es precisamente la motivación de estabilización que incorpora Deep Q-Learning: el *replay buffer* rompe la correlación temporal de las muestras y la red objetivo fija el objetivo de aprendizaje durante un número determinado de pasos, evitando que la red persiga un blanco que ella misma está desplazando.

2.2.3. Entornos

Para abordar este trabajo hemos escogido dos entornos de diferente naturaleza.

Como **entorno discreto** hemos optado por `SimpleGrid`. Este entorno recrea una cuadrícula bidimensional clásica en la que un agente debe desplazarse desde una casilla de inicio hasta una meta esquivando trampas u obstáculos. El agente recibe un estado discreto definido únicamente por sus coordenadas actuales en el tablero, siendo el conjunto de posibles estados finito y, por tanto, ideal para ser abordado mediante métodos tabulares. La **función de recompensa** típica del entorno otorga $+1{,}0$ al agente por alcanzar la casilla objetivo, $-1{,}0$ por caer en una trampa, y una leve penalización de $-0{,}01$ por cada paso dado, incentivando así al agente a encontrar siempre la ruta más rápida posible.

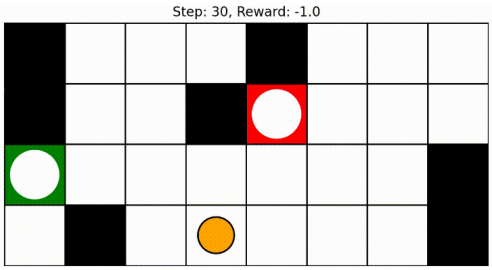

Figura 2.1: Entorno Simplegrid

Nuestro segundo entorno, esta vez **continuo**, es `FlappyBird`. Este entorno recrea el famoso juego móvil Flappy Bird, en el que un pájaro avanza volando horizontalmente teniendo que aletear para sortear obstáculos en forma de tubería. El agente recibe un vector de estado continuo con 12 variables (altura, velocidad e inclinación del pájaro; posición de la tubería anterior y posterior...), siendo el conjunto de posibles estados continuo y, por tanto, inabarcable para un método de estados finitos. La **función de recompensa** propuesta por el repositorio del entorno otorga $+1{,}0$ al agente por atravesar un obstáculo, $+0{,}1$ por mantenerse vivo un frame, $-0{,}5$ por chocarse con el techo y $-1{,}0$ por morir.

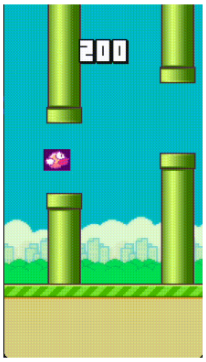

Figura 2.2: Entorno Flappy Bird

2.3. Algoritmos

En esta sección se describen brevemente los fundamentos de los algoritmos de aprendizaje por refuerzo empleados durante la fase experimental.

2.3.1. Monte Carlo

On-Policy

El enfoque on-policy (basado en la política actual) evalúa y mejora de forma simultánea la misma política π que el agente utiliza para tomar decisiones e interactuar con el entorno. Dado que los métodos de Monte Carlo no asumen ningún conocimiento previo de las dinámicas del sistema (son libres de modelo), es estrictamente necesario estimar la función de valor de acción $Q(s, a)$ en lugar de la función de valor de estado $V(s)$. Sin un modelo de transición, el agente necesita conocer el valor explícito de cada decisión posible para poder mejorar su política.

A diferencia de los métodos de diferencias temporales, Monte Carlo requiere que las tareas sean puramente episódicas. El algoritmo no actualiza su conocimiento paso a paso, sino que debe esperar a que el episodio finalice por completo (en el instante T) para observar el retorno total acumulado real, denotado como G_t, a partir de cada instante t:

$$G_t = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{T-1-t} R_T$$

Una vez que el episodio termina, el algoritmo promedia los retornos obtenidos para cada par estado-acción visitado. Si denotamos el número de veces que se ha visitado un par como $N(S_t, A_t)$, la actualización incremental se define como:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{1}{N(S_t, A_t)} \Big[G_t - Q(S_t, A_t) \Big]$$

En entornos no estacionarios, el factor de promediado suele reemplazarse por una tasa de aprendizaje constante α.

Un desafío crítico en el control on-policy es mantener el equilibrio entre la exploración de nuevas trayectorias y la explotación del conocimiento adquirido. Si la política se vuelve completamente codiciosa respecto a sus estimaciones iniciales, el agente podría no visitar nunca ciertas regiones del entorno, perdiéndose la oportunidad de descubrir estrategias superiores. Para evitar esto, habitualmente se emplea una política ϵ-suave, siendo la estrategia ϵ-greedy la más común.

Esta técnica garantiza una exploración continua al asignar una pequeña probabilidad de selección a todas las acciones disponibles. Específicamente, la acción con el valor estimado más alto se selecciona con probabilidad $1-\epsilon+\frac{\epsilon}{|\mathcal{A}(s)|}$, mientras que el resto de las acciones subóptimas se seleccionan de manera uniforme con una probabilidad de $\frac{\epsilon}{|\mathcal{A}(s)|}$. Esto asegura que el agente optimice las decisiones que ya han demostrado ser beneficiosas sin descartar la evaluación asintótica del resto de alternativas.

Off-Policy

El enfoque off-policy (fuera de política) ofrece una solución directa al dilema fundamental entre la exploración y la explotación al desacoplar completamente las políticas del agente. En este paradigma, el algoritmo utiliza dos estrategias distintas y simultáneas: la política objetivo (π), que es la que el agente intenta evaluar y optimizar hacia el ideal (generalmente una política determinista y puramente codiciosa), y la política de comportamiento (b), que es inherentemente exploratoria y es la que se utiliza realmente para interactuar con el entorno y generar los episodios.

Esta separación estructural permite aprender la estrategia óptima subyacente mientras el agente sigue explorando de forma ininterrumpida. Sin embargo, para que el aprendizaje cruzado sea teóricamente posible, debe cumplirse la suposición de cobertura: toda acción que la política objetivo π pueda llegar a tomar, debe tener una probabilidad estrictamente mayor a cero de ser elegida por la política de comportamiento b (es decir, $b(a|s) > 0$ siempre que $\pi(a|s) > 0$).

Dado que los retornos observados al final del episodio (G_t) se han generado siguiendo las probabilidades de la política b, promediarlos directamente para actualizar las estimaciones de π introduciría un sesgo estadístico severo. Para corregir esta discrepancia de distribuciones, se aplica un mecanismo estadístico avanzado conocido como muestreo de importancia (*importance sampling*). Este método pondera los retornos obtenidos según la probabilidad relativa de que la trayectoria observada bajo b hubiera sido ejecutada exactamente igual bajo π.

El ratio de muestreo de importancia desde el instante temporal t hasta el estado terminal en T se define como el producto secuencial de estas probabilidades relativas:

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k \mid S_k)}{b(A_k \mid S_k)} \tag{2.1}$$

Para actualizar la función de valor, el retorno real G_t se escala multiplicándolo por este ratio $\rho_{t:T-1}$. Dependiendo del método de actualización, surgen dos variantes: el muestreo de importancia ordinario, que promedia de forma simple y suele sufrir de una varianza extrema e ilimitada (debido a que la multiplicación en cadena de probabilidades genera valores drásticamente altos o bajos), y el muestreo de importancia ponderado, que normaliza las actualizaciones reduciendo drásticamente la varianza a costa de introducir un pequeño sesgo inicial. En implementaciones prácticas, la variante ponderada resulta fundamental para asegurar la estabilidad y convergencia del algoritmo.

2.3.2. SARSA

SARSA constituye un método de aprendizaje por Diferencias Temporales (TD) enfocado en el control. Se clasifica estrictamente como un algoritmo *on-policy*, lo que significa que evalúa y mejora de forma simultánea la política que el agente está ejecutando para interactuar con el entorno. A diferencia de los métodos *off-policy*, SARSA aprende el valor exacto de la política de comportamiento, incorporando inherentemente los pasos de exploración (como las acciones aleatorias en una política ϵ-greedy) dentro de sus estimaciones de valor.

Su nombre es un acrónimo que proviene de la secuencia exacta de eventos continuos que el algoritmo utiliza para realizar cada actualización: Estado actual (S_t), Acción actual elegida (A_t), Recompensa inmediata recibida (R_{t+1}), siguiente Estado observado (S_{t+1}) y, finalmente, la siguiente Acción elegida (A_{t+1}) de acuerdo a la política vigente.

La regla de actualización matemática que rige el aprendizaje en SARSA se define formalmente como:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\Big]$$

En esta ecuación, α representa la tasa de aprendizaje y γ el factor de descuento. El término $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$ actúa como el objetivo hacia el cual se ajusta la estimación. Dado que este objetivo depende explícitamente de la acción A_{t+1} que el agente efectivamente tomará, SARSA tiende a aprender políticas más conservadoras y seguras. Si explorar cerca de un estado de alto riesgo puede llevar a penalizaciones severas, SARSA ajustará los valores Q a la baja para evitar esa región, reduciendo su propia tendencia a explorar.

A diferencia de los métodos de Monte Carlo, que requieren esperar estrictamente al final de un episodio para observar el retorno total acumulado (G_t) antes de aprender, SARSA aplica el principio de *bootstrapping*. Esto significa que actualiza sus estimaciones paso a paso basándose parcialmente en otras estimaciones previas (el valor de $Q(S_t, A_t)$). Esta característica fundamental permite que el aprendizaje ocurra de manera incremental y continua, resultando especialmente ventajoso en episodios extremadamente largos o en tareas continuas que carecen de un estado terminal.

2.3.3. Q-Learning

Q-Learning representa el algoritmo fundamental de aprendizaje por Diferencias Temporales (TD) enfocado en el control de tipo *off-policy*. Introducido originalmente por Watkins en 1989 [10], este método supuso un avance crucial en el campo del Aprendizaje por Refuerzo, ya que busca aproximar directamente la función de valor de acción óptima, denotada como

Q^*, independientemente de las acciones exploratorias que el agente esté realizando en ese momento bajo su política de comportamiento.

Para lograr este desacoplamiento entre la política objetivo que se optimiza y la política que interactúa con el entorno, Q-Learning incorpora el operador de maximización (máx) sobre las posibles acciones del estado siguiente (S_{t+1}) dentro de su regla de actualización:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\Big]$$

En esta expresión matemática, el objetivo queda definido por el término $R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$. De esta forma, el algoritmo evalúa la transición actual asumiendo de manera optimista que en el futuro siempre se tomará la mejor decisión posible (la acción codiciosa que maximiza la estimación de Q). Esto ocurre de forma totalmente independiente a la acción real que el agente termine ejecutando en el siguiente paso, la cual podría ser aleatoria o subóptima si está bajo una estrategia de exploración como ϵ-greedy.

Esta característica *off-policy* contrasta fuertemente con algoritmos como SARSA. Mientras que SARSA aprende a alejarse de zonas de alto riesgo penalizando estados cercanos al peligro (porque reconoce que su propia exploración podría hacerle fallar), Q-Learning ignora los riesgos derivados de la exploración temporal y traza directamente la ruta ideal y más corta hacia la recompensa.

Bajo las condiciones teóricas estándar de aproximación estocástica; es decir, asumiendo que todos los pares estado-acción continúan siendo visitados repetidamente y que la tasa de aprendizaje α decae de manera apropiada; esta actualización garantiza una convergencia estable hacia la estrategia óptima con probabilidad uno.

2.3.4. SARSA Semi-Gradiente

Para resolver problemas en entornos continuos, el espacio de estados es infinito (o gigantesco), por lo que es imposible almacenar los valores en una tabla clásica. En su lugar, se adapta SARSA, parametrizando la función de valor de acción Q mediante un vector de pesos $\mathbf{w}$, denotada ahora como $\hat{q}(s, a, \mathbf{w})$.

Al hacer esto, actualizar el conocimiento del agente ya no significa cambiar el valor de una celda en una tabla, sino ajustar los pesos $\mathbf{w}$ para modificar la forma de la función Q en todo el entorno continuo. La ecuación de actualización es:

$$\begin{aligned}\mathbf{w} \leftarrow \mathbf{w} + \alpha\Big[&R_{t+1} + \gamma\hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}) \\ &- \hat{q}(S_t, A_t, \mathbf{w})\Big]\nabla_{\mathbf{w}}\hat{q}(S_t, A_t, \mathbf{w})\end{aligned}$$

Desglosando la actualización de la función Q:

- $\mathbf{w}$: Pesos que definen a la función Q para el entorno continuo.
- $\hat{q}(S_t, A_t, \mathbf{w})$: Nuestra estimación actual de la función Q.
- En el corchete se mide la diferencia entre nuestra estimación y el objetivo temporal ($R_{t+1} + \gamma\hat{q}$).
- $\nabla_{\mathbf{w}}\hat{q}$: El gradiente de la función Q. Nos indica matemáticamente cómo mover los pesos $\mathbf{w}$ para corregir el error en nuestra estimación.

Al derivar la función para actualizar los pesos, notamos que el objetivo de nuestra actualización ($R_{t+1} + \gamma\hat{q}(S_{t+1}, A_{t+1}, \mathbf{w})$) también depende de los propios pesos $\mathbf{w}$.

Se llama semi-gradiente porque hacemos una simplificación intencionada: ignoramos esta dependencia y tratamos al objetivo como una constante fija. Solo calculamos el gradiente de nuestra estimación actual.

Algorithm 5 SARSA Semi-gradiente para entornos continuos

Require: $\alpha > 0$, $\gamma \in [0, 1]$, pesos iniciales $\mathbf{w}$
for cada episodio **do**
  Inicializar S y elegir acción A usando política sobre $\hat{q}$
  while S no sea terminal **do**
    Ejecutar A, ver recompensa R y nuevo estado S'
    Elegir siguiente acción A' desde S'
    if S' es terminal **then**
      $Obj \leftarrow R$
    else
      $Obj \leftarrow R + \gamma\hat{q}(S', A', \mathbf{w})$
    end if
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[Obj - \hat{q}(S, A, \mathbf{w})\big]\nabla_{\mathbf{w}}\hat{q}(S, A, \mathbf{w})$
    $S \leftarrow S'$, $A \leftarrow A'$
  end while
end for

2.3.5. Deep Q-Learning (DQN)

Deep Q-Learning extiende los principios de Q-Learning para operar en entornos con espacios de estados continuos y de alta dimensionalidad, aproximando la función de acción-valor óptima mediante una red neuronal profunda parametrizada por θ, de modo que $Q(s, a; \theta) \approx q^*(s, a)$. El objetivo de aprendizaje en cada paso es minimizar el error de diferencias temporales entre la estimación actual y el valor objetivo:

$$\mathcal{L}(\theta) = \mathbb{E}\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta\right)^2\right]$$

donde θ^- son los pesos de la red objetivo, distintos de θ. La actualización de θ se realiza mendiante descenso del gradiente estocástico sobre esta pérdida.

El aprendizaje directo con redes neuronales sobre secuencias de experiencia correlacionadas temporalmente es inherentemente inestable. Para resolverlo, DQN incorpora dos mecanismos fundamentales.

- ***Replay buffer***. Un *buffer* [11] de capacidad fija almacena las transiciones $(s, a, r, s', \text{done})$ generadas durante la interacción con el entorno. En cada paso de entrenamiento, se extrae un lote aleatorio de transiciones del *buffer* en lugar de utilizar la transición más reciente. Esto rompe la correlación temporal entre muestras consecutivas, convirtiendo el problema en uno más similar al aprendizaje supervisado estándar.

- **Target Network**. Una segunda red neuronal, parametrizada por θ^-, se mantiene congenlada durante N pasos y proporciona el valor objetivo $r + \gamma \max_{a'} Q(s', a'; \theta^-)$. Sin este mecanismo, la red Q_θ perseguiría un objetivo que ella misma desplaza en cada actualización, creando un bucle de retroalimentación positiva que produce sobreestimación catastrófica de los valores Q. Cada N pasos, los pesos de la red principal se copian a la objetivo.
- **Double Deep Q-Learning (DDQN)**. DQN estándar sufre de un sesgo de sobreestimación inherente al operador máx. Cuando los errores de estimación son independientes con media cero, la desigualdad de Jensen garantiza que $\mathbb{E}[\max_i X_i] \geq \max_i \mathbb{E}[X_i]$: el máximo de estimaciones ruidosas sobreestima sistemáticamente el máximo de los valores reales. Double DQN [12] mitiga este sesgo desacoplando la selección de la acción de su evaluación: la red principal Q_θ selecciona la acción más prometedora y la red objetivo Q_{θ^-} estima de forma independiente el valor Q de esa acción. El objetivo es entonces:

$$r + \gamma Q\left(s', \arg\max_{a'} Q(s', a'; \theta); \theta^-\right)$$

2.4. Métodos Tabulares

2.4.1. Entorno Simple Grid

Para aumentar la complejidad y evaluar la capacidad de generalización, el entorno no utiliza un mapa fijo. En su lugar, se genera una cuadrícula procedural de 4x4 en cada episodio con una probabilidad del 25 % de aparición de obstáculos. Para evitar penalizar al agente en laberintos sin salida, se aplica un algoritmo de Búsqueda en Anchura (BFS) que garantiza que el mapa generado siempre tenga solución.

2.4.2. Configuración Experimental

Se han evaluado cuatro algoritmos: SARSA, Q-Learning, Monte Carlo On-Policy y Monte Carlo Off-Policy. Todos ellos se entrenan durante 1500 episodios, compartiendo hiperparámetros base como el factor de descuento ($\gamma = 0{,}99$) y una tasa de exploración constante ($\epsilon = 0{,}1$). Para los métodos de diferencias temporales, se utiliza una tasa de aprendizaje $\alpha = 0{,}1$. Además, se impone un límite de 100 pasos por episodio para evitar bucles infinitos durante la fase de exploración.

2.4.3. Métricas de Desempeño y Resultados

El rendimiento de los modelos se ha medido utilizando tres métricas principales: la recompensa total acumulada, la longitud del episodio (número de pasos hasta la meta) y la tasa de éxito. Para visualizar claramente la tendencia de aprendizaje subyacente y reducir el ruido natural de la exploración, los datos de las gráficas se han suavizado mediante una media móvil de los últimos 50 episodios.

La siguiente figura agrupa la evolución temporal del rendimiento de los distintos métodos, permitiendo identificar rápidamente cuáles logran descifrar el entorno dinámico.

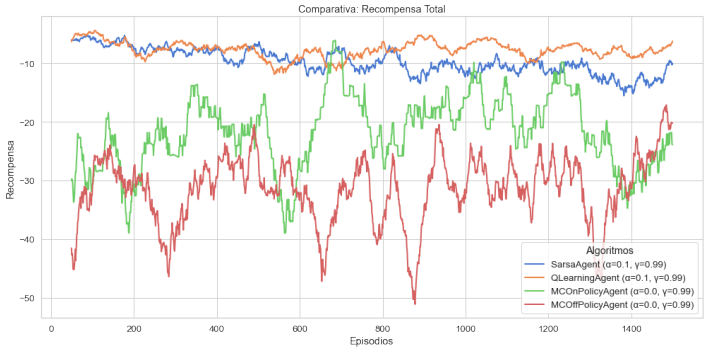

Figura 2.3: Comparativa del rendimiento global de los métodos tabulares en mapas generados proceduralmente.

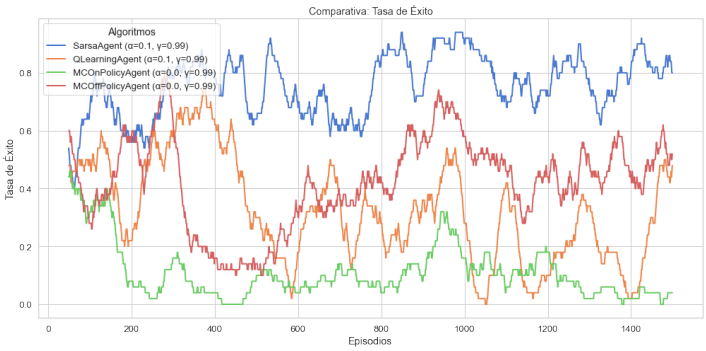

Figura 2.4: Comparativa de la tasa de éxito de los métodos tabulares en mapas generados proceduralmente.

Al analizar conjuntamente la evolución de la recompensa y la tasa de éxito (Figuras 2.3 y 2.4), se observa una clara diferencia en el comportamiento de los algoritmos evaluados.

En la familia de diferencias temporales, **Q-Learning** (línea naranja) obtiene la mayor recompensa media, manteniéndose relativamente estable y cercano a los valores menos negativos durante gran parte del entrenamiento. Sin embargo, su tasa de éxito es altamente inestable, mostrando oscilaciones pronunciadas y periodos en los que el rendimiento cae significativamente.

Por otro lado, **SARSA** (línea azul) presenta una recompensa ligeramente inferior, pero mantiene una tasa de éxito mucho más alta y estable, situándose frecuentemente entre 0.7 y 0.9. Esto indica que, aunque su recompensa no sea la mejor, logra resolver el entorno con mayor consistencia.

En cuanto a los métodos de **Monte Carlo**, los resultados muestran un rendimiento claramente inferior. La variante **MC On-Policy** (línea verde) mantiene tanto la recompensa como la tasa de éxito en niveles muy bajos durante todo el entrenamiento, lo que indica una incapacidad para aprender una política eficaz en el entorno.

Finalmente, **MC Off-Policy** (línea roja) muestra cierto aprendizaje a lo largo del entrenamiento, alcanzando tasas de éxito moderadas en algunos periodos. No obstante, su recompensa acumulada sigue siendo la peor de todos los algoritmos y presenta una elevada variabilidad, lo que evidencia una política poco eficiente.

2.4.4. Análisis

La comparación entre las dos familias de algoritmos pone de manifiesto la importancia de la actualización incremental en entornos dinámicos.

Los métodos de **Monte Carlo** (MC) resultan menos adecuados para este tipo de problema. La variante **MC On-Policy** apenas consigue aprender una estrategia útil, manteniéndose durante todo el entrenamiento con tasas de éxito cercanas a cero. Por su parte, **MC Off-Policy** logra mejorar parcialmente su rendimiento, pero presenta una elevada varianza y una recompensa acumulada muy baja. Esto sugiere que esperar al final de cada episodio para actualizar los valores introduce demasiado retraso en la señal de aprendizaje.

En contraste, los métodos de diferencias temporales (TD) muestran un comportamiento mucho más efectivo. **Q-Learning** consigue maximizar mejor la recompensa esperada, pero su política es inestable y su tasa de éxito fluctúa considerablemente a lo largo del entrenamiento.

Por el contrario, **SARSA** demuestra ser el algoritmo más estable y consistente, manteniendo tasas de éxito elevadas durante la mayor parte del proceso de aprendizaje. Al actualizar sus valores siguiendo la política que realmente está ejecutando, tiende a adoptar un comportamiento más prudente y robusto frente a la variabilidad del entorno.

2.5. Métodos Aproximados

2.5.1. Entorno Flappy Bird

Sobre la función de recompensa base, introducimos un *wrapper* de *reward shaping* que añade una bonificación $r_{\alpha,t} \in (0, \alpha)$ proporcional a la proximidad vertical del pájaro al centro del hueco de la tubería siguiente:

$$q_t = q_{t,\text{base}} + r_{\alpha,t}$$

La intuición de esta heurística es que un pájaro que se mantiene alineado con el hueco necesitará menos correcciones bruscas para atravesarlo, lo que debería acelerar la convergencia.

2.5.2. Configuración Experimental

SARSA Semi-Gradiente

Se han definido dos estrategias diferentes construidas a partir de una misma base. Inicialmente, se podría plantear aproximar función Q como tensor n-dimensional en el consultar valores según una discretización específica de las variables del entorno. Esta aproximación resulta inviable ya que puede dar lugar a tensores enormes si se divide el espacio continuo en muchas zonas discretas; o a imprecisión si no se realizan las suficientes divisiones necesarias para representar correctamente un espacio continuo. Por ello, decidimos utilizar un pequeño MLP de 2 capas ocultas de 64 neuronas que aprende a lo largo del tiempo a aproximar la función Q.

A partir de esta base, definimos el agente básico de SARSA semi-gradiente al que entrenamos para superar obstáculos. Cómo método adicional, decidimos añadir un sesgo off-policy mediante un buffer de repetición [11], en el que se almacenarán las diferentes transiciones que van ocurriendo, de esta forma, podemos entrenar a la red de aproximación de Q utilizado muestras aleatorias de dicho buffer, limitando la correlación temporal de las muestras utilizadas.

Las dos variantes del agente de SARSA semi-gradiente han sido entrenadas en 10000 episodios de simulación, estableciendo un umbral de de exploración bajo decrementado linealmente con el progreso del entrenamiento ($\epsilon = [0{,}1, 0{,}05]$).

Deep Q-Learning

El diseño experimental evalúa seis variantes algorítmicas que aíslan de forma controlada tres componentes críticos: el uso de red objetivo, la aplicación de *reward shaping* y el uso de Double DQN. La tabla siguiente resume las configuraciones:

ID	Estrategia	T. Network	DDQN	α (shaping)
A	Vanilla DQN	No	No	0.00
B	DQN Estándar	Sí	No	0.00
C	DQN + S. leve	Sí	No	0.05
D	DQN + S. moderado	Sí	No	0.25
E	DQN + S. agresivo	Sí	No	0.50
F	Double DQN	Sí	Sí	0.00

Tabla 2.1: Configuraciones experimentales evaluadas en el estudio de Deep Q-Learning.

La Estrategia A actúa como línea base pura sin ningún mecanismo de estabilización, permitiendo cuantificar empíricamente el impacto de la red objetivo. La Estrategia B es la línea base moderna. Las Estrategias C, D y E estudian el efecto del *reward shaping* a distintas intensidades. La Estrategia F evalúa si el desacoplamiento selección-evaluación de Double DQN produce políticas más robustas.

Todos los hiperparámetros no relacionados con la variante se mantienen constantes entre estrategias:

Hiperparámetro	Valor
Episodios de entrenamiento	2 000
Semillas por estrategia	5
Capacidad del replay buffer	10 000
Mínimo de experiencias antes de entrenar	1 000
Tamaño del mini-lote	64
Neuronas por capa oculta (2 capas)	128
Tasa de aprendizaje (α)	10^{-3}
Factor de descuento (γ)	0,99
ϵ inicial / mínimo / decaimiento	1,0 / 0,01 / 0,996
Frecuencia de actualización de la red objetivo	500 pasos
Pasos máximos por episodio	2 000

Tabla 2.2: Hiperparámetros compartidos por todas las estrategias DQN.

El proceso de evaluación consta de dos fases para cada semilla:

- **Entrenamiento**. Cada agente se entrena durante 2000 episodios registrando la recompensa acumulada, la longitud del episodio, la pérdida y los valores Q máximos. Durante el entrenamiento se aplica un mecanismo de *checkpointing*: se guardan los pesos de la red cada vez que la puntuación real media (medida en tuberías superadas, sin incluir el bonus de shaping) supera el mejor valor registrado en una ventana deslizante de 30 episodios. Esto garantiza que la política presentada sea la mejor encontrada durante el entrenamiento, evitando que fluctuaciones tardías degraden el resultado final.

- **Evaluación**. Inmediatamente tras el entrenamiento, se carga el mejor modelo guardado, se fija $\epsilon = 0$ y el agente se evalúa durante 100 episodios sobre el entorno sin shaping. Esto permite una comparación directa y justa entre todas las estrategias, incluyendo las que utilizaron recompensas modificadas durante el entrenamiento, ya que todas se miden sobre la misma señal de recompensa nativa.

Dado que cada estrategia se ejecuta con 5 semillas y se evalúa en 100 episodios por semilla, se obtienen 500 puntuaciones por estrategia sobre las que se calcula la media, mediana, desviación típica y máximo.

Una vez concluido el estudio exploratorio, las dos estrategias con mejor rendimiento se reentrenan durante 5000 episodios con las mismas 5 semillas y la misma metodología de evaluación. El objetivo es determinar si la ventaja estadística observada a 2000 episodios se mantiene o invierte con entrenamiento prolongado, y en particular si Double DQN, al producir estimaciones Q más conservadoras, se beneficia proporcionalmente más del entrenamiento extendido que DQN estándar.

2.5.3. Métricas de Desempeño y Resultados

Para el análisis de resultados en métodos continuos, dado que estamos en un entorno de juego, se han utilizado como métricas principales la recompensa acumulada durante la partida y la cantidad total de frames que nuestro pájaro ha sobrevivido, relacionada esta última directamente con la cantidad de obstáculos que se han superado.

SARSA Semi-Gradiente

En SARSA Semi-Gradiente compararemos en los dos agentes propuestos las métricas expuestas anteriormente. Además, utilizando los resultados de entrenamiento, se han realizado simulaciones con $\epsilon = 0$ para probar el comportamiento de los agentes únicamente explotando el comportamiento aprendido.

Véase que el agente SARSA Semi-Gradiente puramente on-policy no logra aprender en ningún momento los patrones necesarios para poder esquivar obstáculos. En la figura 2.5 no se aprecia una tendencia de crecimiento clara ni en la gráfica de recompensa acumulada ni en la de frames sobrevividos.

Por el contrario, el agente con sesgo off-policy introducido mediante el buffer de repetición presenta varias características diferenciadoras que evidencian un mejor aprendizaje en la tarea de esquivar obstáculos. Lo más significativo es la estabilidad que presenta entre episodios; en comparación con su contra parte on-policy, este agente presenta mucha menos varianza en las diferentes métricas entre episodios.

Se muestra perfectamente como conforme avanza el proceso de entrenamiento el crecimiento de recompensa acumulada y frames sobrevividos es lineal y estable. Nótese que incrementamos la penalización de pérdida con el paso de los episodios, penalizando al agente de forma proporcional al frame en el que falló su tarea.

La diferencia encontrada entre los agentes se debe a la forma en la que estamos entrenando a la red de aproximación de Q en el proceso de entrenamiento. La versión on-policy pura entrena a la red con las decisiones inmediatas que toma en el momento. Debido al pequeño tamaño del MLP y la cantidad de variables del entorno, se está introduciendo un sesgo de correlación temporal en la red, provocando que conforme avanza un episodio, el MLP olvide lo aprendido al inicio del mismo o en episodios de entrenamiento anteriores. Como solución se barajó incrementar el tamaño del MLP, pero eso podría no ser viable en ciertos entornos de despliegue realista, por lo que nos decantamos por el buffer de repetición.

La figura 2.7 muestra como, una vez entrenados, cuando aplicamos explotación pura, nuestra versión sesgada off-policy supera con creces a la versión de SARSA Semi-gradiente pura, ya que esta última no es capaz de cumplir su tarea en ningún episodio mientras que utilizando un buffer de repetición si es posible superar 85 tuberías de media por episodio, sobreviviendo más de 3000 frames.

Llegamos a la conclusión de que para tareas tan complejas como la que presenta nuestro entorno, utilizar métodos puramente on-policy es inviable, siendo necesaria información relativa a otras decisiones que se han tomado a lo largo del tiempo para facilitar tomar una decisión acertada en el presente.

Deep Q-Learning

Para la comparación directa entre estrategias se utiliza exclusivamente la puntuación real obtenida en la fase de evaluación greedy (tuberías superadas en el entorno sin shaping), descartando las métricas de recompensa interna para las estrategias C, D y E, cuyas magnitudes no son comparables con las de A, B y F al incluir el bonus de shaping.

Estudio exploratorio (2 000 episodios). La siguiente tabla recoge el resumen estadístico del rendimiento real de las seis estrategias (500 evaluaciones por estrategia: 100 episodios $\times$ 5 semillas):

Estrategia	Media	Mediana	Desv. típica	Máximo
B - DQN Estándar	4,41	2,0	6,08	52
F - Double DQN	3,54	2,0	4,75	37
C - Shaping leve	2,72	1,0	3,61	34
D - Shaping moderado	2,68	2,0	3,19	17
A - Vanilla DQN	1,54	1,0	2,57	23
E - Shaping agresivo	1,41	1,0	1,56	9

Tabla 2.3: Rendimiento determinista final en el estudio exploratorio (100 episodios por semilla, entorno sin shaping). Ordenado por media de tuberías superadas.

El ranking por media es: B (4,41) $>$ F (3,54) $>$ C (2,72) $\approx$ D (2,68) $\gg$ A (1,54) $>$ E (1,41).

Análisis de cada componente experimental.

- *Impacto de la Target Network (A vs. B).* La omisión de la red objetivo (Agente A) provoca una sobreestimación catastrófica de los valores Q, con picos superiores a 450 alrededor del episodio 300 (banda de confianza por encima de 700), seguidos de un colapso del aprendizaje. El

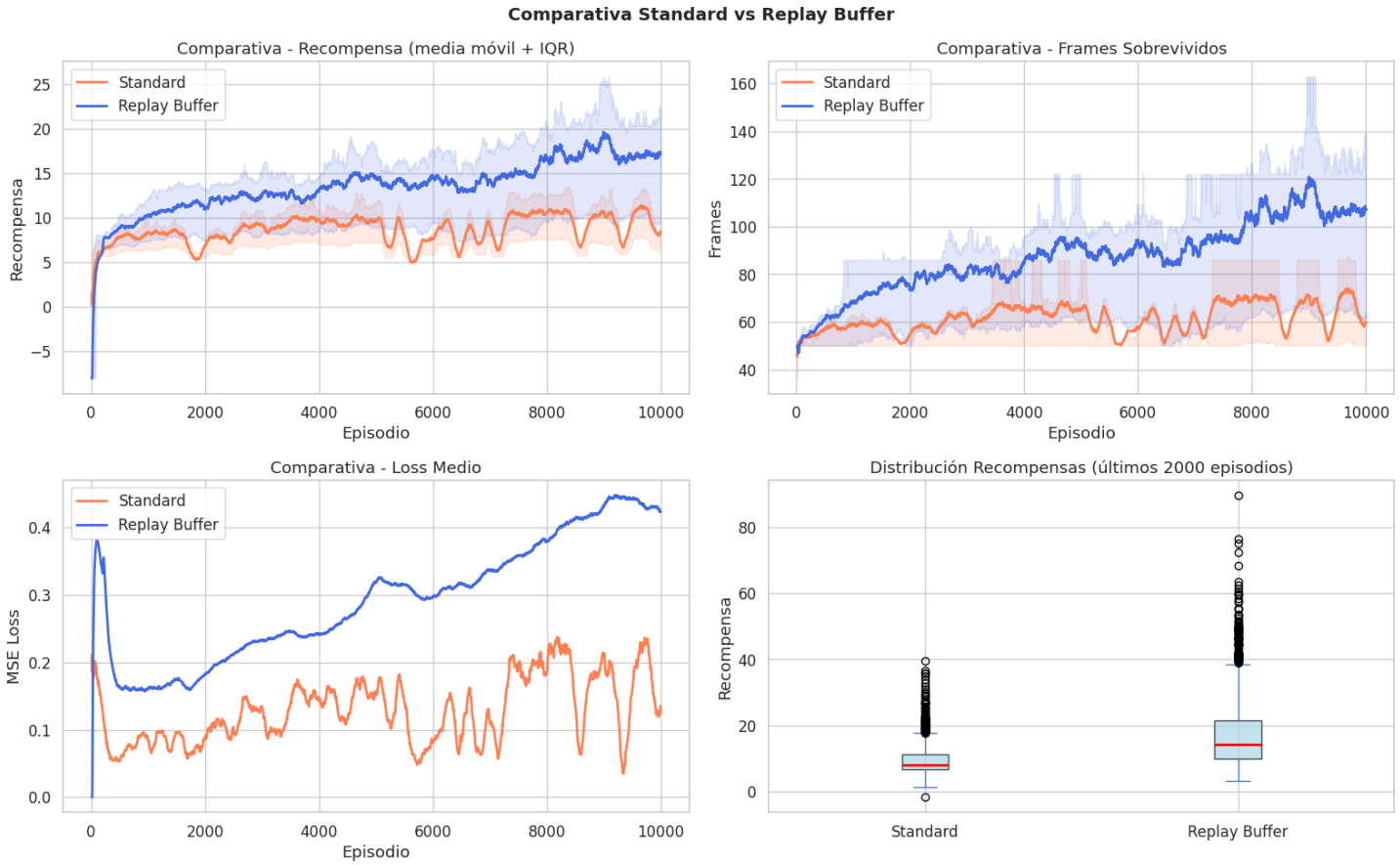

Figura 2.5: Métricas de entrenamiento SARSA Semi-Gradiente

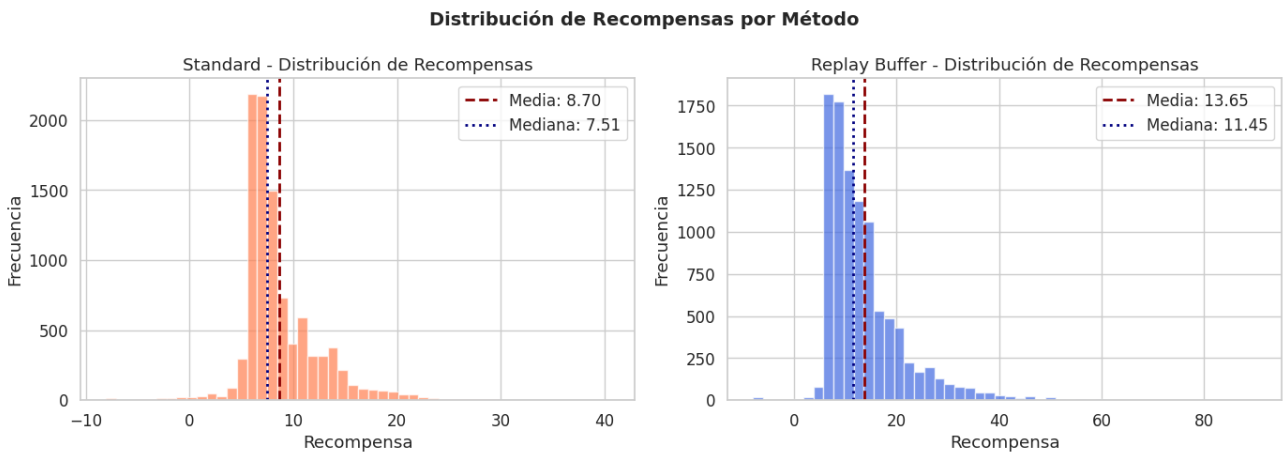

Figura 2.6: Distribución de recompensas durante el entrenamiento SARSA Semi-Gradiente

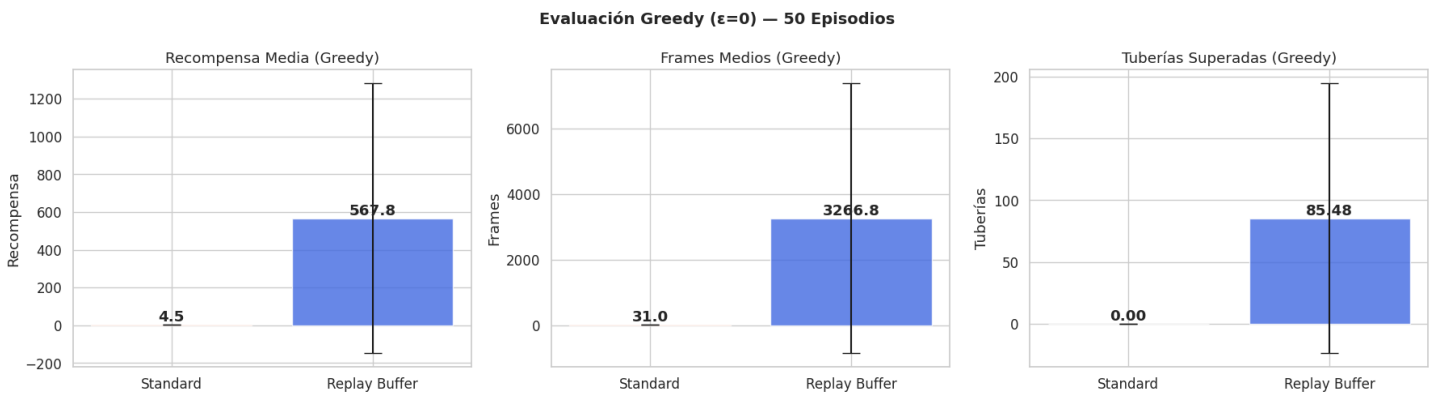

Figura 2.7: Métricas de explotación SARSA Semi-Gradiente

mecanismo del fallo es el siguiente: sin red objetivo, la misma red Q_θ se usa tanto para seleccionar la mejor acción como para estimar su valor. Cuando θ se actualiza, el objetivo $r + \gamma \max_{a'} Q_\theta(s', a')$ cambia simultáneamente, creando un bucle de retroalimentación positiva. La red objetivo Q_{θ^-} rompe este ciclo fijando el objetivo durante N pasos, estabilizando los valores Q por debajo de 15 en todas las estrategias restantes. El impacto sobre el rendimiento final es directo: A obtiene solo 1,54 tuberías de media frente a las 4,41 de B.

- *Efecto del Reward Shaping (C, D, E).* Las tres variantes de shaping obtienen rendimientos inferiores a DQN estándar. El operador $\max_{a'} Q(s', a')$ produce un sesgo positivo inherente: cuando los errores de estimación tienen media cero, la desigualdad de Jensen garantiza que $\mathbb{E}[\max_i X_i] \geq \max_i \mathbb{E}[X_i]$. El shaping aditivo amplifica este efecto al inflar los retornos observados, empujando a la red a producir predicciones Q más altas. A mayor α, mayor la distorsión. Además, el shaping empleado no es *potential-based* en el sentido de Ng, Harada y Russell [13], ya que no tiene la forma $F(s, a, s') = \gamma\Phi(s') - \Phi(s)$, y por tanto no preserva la política óptima. Con $\alpha = 0{,}50$ (Agente E), el bonus por frame (+0,50) domina la recompensa por tubo (+1,00), produciendo un agente que optimiza flotar centrado en lugar de cruzar tuberías (*reward misalignment*), lo que explica su último puesto con solo 1,41 tuberías de media.
- *Double DQN vs. DQN Estándar (F vs. B).* Para determinar si la diferencia entre B y F es estadísticamente significativa, aplicamos tests U de Mann-Whitney unilaterales sobre las 500 puntuaciones de cada estrategia. Esta prueba no paramétrica es adecuada dado que las distribuciones de puntuación en Flappy Bird no son normales. Los resultados se recogen en la siguiente tabla:

Hipótesis	Estadístico U	p-valor	Resultado
H_1: F > B	118 748,5	0,9195	No significativo
H_1: F > C	142 864,0	≈ 0	Significativo
H_1: B > C	146 430,5	≈ 0	Significativo

Tabla 2.4: Tests de Mann-Whitney U unilaterales ($\alpha = 0{,}05$) sobre las puntuaciones de evaluación greedy del estudio exploratorio.

B y F son estadísticamente indistinguibles a 2 000 episodios ($p = 0{,}92$). El p-valor alto en la dirección F > B sugiere incluso una tendencia contraria: B tiende a superar a F, aunque sin alcanzar significancia ($p_{\mathrm{B>F}} \approx 0{,}08$). Ambas estrategias superan significativamente a C, D y E ($p \approx 0$). Dado este empate estadístico, se seleccionan B y F como finalistas para el estudio extendido.

Estudio extendido (5 000 episodios). Al prolongar el entrenamiento, el ranking se invierte:

Estrategia	Media	Mediana	Desv. típica	Máximo	Media (2 000 ep.)	Variación
F — Double DQN	4,97	2,0	7,24	52	3,54	+41 %
B — DQN Estándar	3,73	1,0	5,95	52	4,41	−15 %

Tabla 2.5: Rendimiento determinista final en el estudio extendido (5 000 episodios), comparado con el exploratorio.

Double DQN mejora un 41 % respecto a su rendimiento exploratorio, mientras que DQN estándar retrocede un 15 %. Este resultado confirma la hipótesis teórica: al desacoplar la selección de la acción de su evaluación, Double DQN acumula menos sesgo de sobreestimación y se beneficia proporcionalmente más del entrenamiento prolongado. El retroceso de B sugiere que DQN estándar sufre de *overfitting* temporal: encuentra buenas políticas en ventanas intermedias del entrenamiento, pero las pierde con episodios adicionales debido a la sobreestimación acumulativa que corrompe gradualmente la función Q. La figura **??** muestra la evolución de la pérdida en el estudio extendido para B y F, donde se aprecia la mayor estabilidad de F a lo largo de los 5 000 episodios.

Comparativa de métodos aproximados

Experimentalmente, se ha observado que para este entorno en específico, nuestra versión de SARSA Semi-Gradiente sesgada ha funcionado mucho mejor que los métodos de Deep Q-Learning propuestos. Numéricamente, nuestra mejor versión de Deep Q-Learning supera una media de 4,41 tuberías, mientras que SARSA-SG más de 85.

Esto se puede justificar atendiendo a la naturaleza de ambos algoritmos. DQN es inherentemente optimista; sobre todo al principio del entrenamiento, la red comete errores, y algunas acciones no óptimas podrían tener valores altos asociados, lo que implica que el operador de maximización seleccionaría dichas acciones sobre otras que proporcionarían mejores resultados a largo plazo.

SARSA-SG es más precavido, y, con la introducción del sesgo del buffer de repetición, y al evaluar la política con exploración incluida, se rompe la correlación de los datos secuenciales y se obtiene una convergencia más suave hacia comportamientos que resuelven satisfactoriamente la tarea.

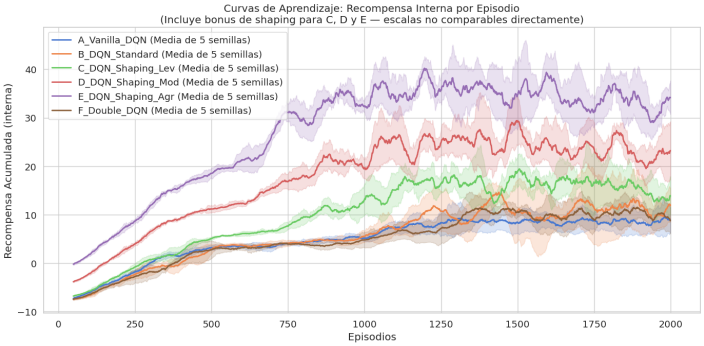

Figura 2.8: Curvas de recompensa acumulada Deep Q-Learning

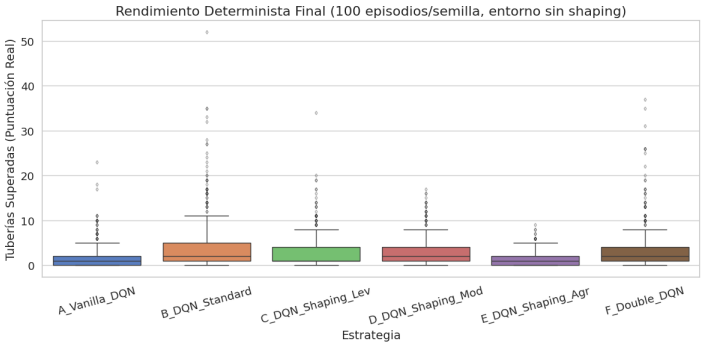

Figura 2.12: Rendimiento de cada agente en Deep Q-Learning

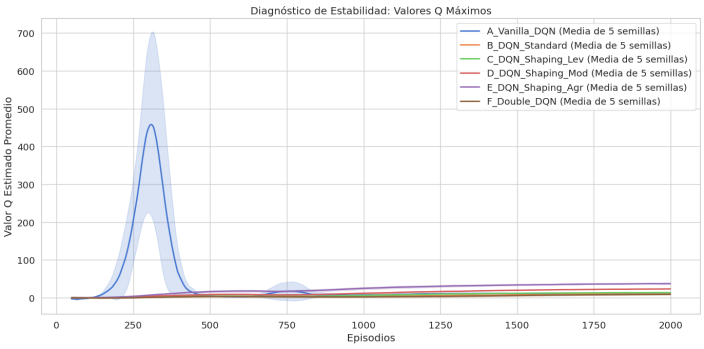

Figura 2.9: Q valor predicho Deep Q-Learning

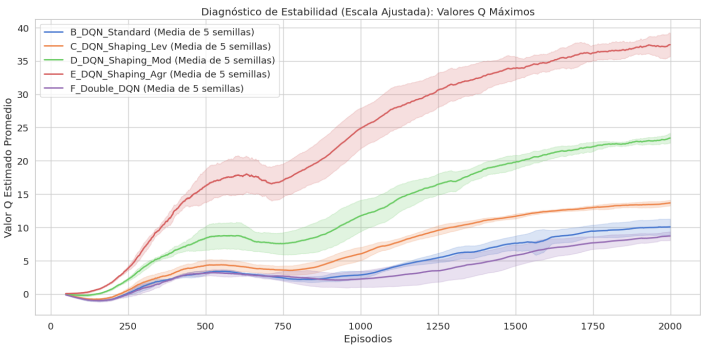

Figura 2.13: Q valor predicho Deep Q-Learning (Sin estrategia A)

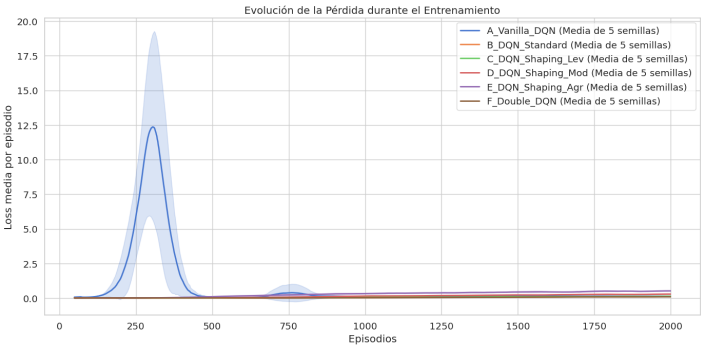

Figura 2.10: Loss Deep Q-Learning

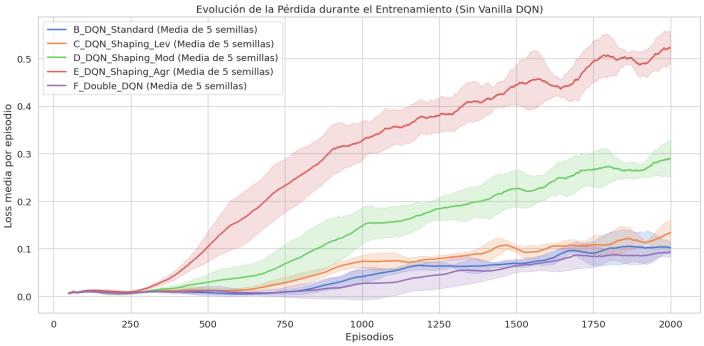

Figura 2.14: Loss Deep Q-Learning (Sin estrategia A)

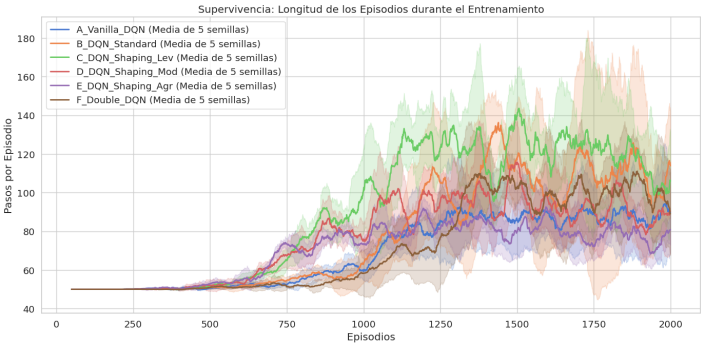

Figura 2.11: Longitud de los episodios Deep Q-Learning

Bibliografía

[1] Robbins, H. (1952). Some aspects of the sequential design of experiments. *Bulletin of the American Mathematical Society*, *58*(5), 527–535. https://doi.org/10.1090/S0002-9904-1952-09620-8

[2] Lai, T. L., & Robbins, H. (1985). Asymptotically efficient adaptive allocation rules. *Advances in Applied Mathematics*, *6*(1), 4–22. https://doi.org/10.1016/0196-8858(85)90002-8

[3] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.

[4] Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, *47*(2–3), 235–256. https://doi.org/10.1023/A:1013689704352

[5] Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, *47*(2–3), 235–256. https://doi.org/10.1023/A:1013689704352

[6] Luce, R. D. (1959). *Individual choice behavior: A theoretical analysis*. Wiley.

[7] Kuleshov, V., & Precup, D. (2014). Algorithms for the multi-armed bandit problem. *Journal of Machine Learning Research*, *1*, 1–48. https://arxiv.org/abs/1402.6028

[8] Thompson, W. R. (1933). On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, *25*(3–4), 285–294. https://doi.org/10.2307/2332286

[9] Russo, D. J., Van Roy, B., Kazerouni, A., Osband, I., & Wen, Z. (2018). A tutorial on Thompson sampling.*Foundations and Trends in Machine Learning*, *11*(1), 1–96. https://doi.org/10.1561/2200000070

[10] Watkins, C. J. C. H. (1989). *Learning from Delayed Rewards* (Ph.D. thesis). King's College, Cambridge. Available at: https://www.cs.rhul.ac.uk/~chrisw/thesis.html

[11] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533. DOI: https://doi.org/10.1038/nature14236

[12] Van Hasselt, H., Guez, A., & Silver, D. (2016). Deep Reinforcement Learning with Double Q- Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence* (Vol. 30, No. 1). Available at: https://arxiv.org/abs/1509.06461

[13] Ng, A. Y., Harada, D., & Russell, S. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning* (pp. 278–287). Available at: https://people.eecs.berkeley.edu/~russell/papers/icml99-shaping.pdf

[14] Uso de agentes LLM para la asistencia en la elaboración de texto, toma de ideas y asistencia en la escritura de código.